Brooklyn Muñoz Acevedo

AJEDREZ PYTHON

PROMPT:

Hola qué tal, tengo que hacer un trabajo para Entornos de Desarrollo.

Tengo que crear un Ajedrez en Python. Ya tengo diseñado el modelo y necesito que me generes el esqueleto en Python (todas las clases) según lo que yo te pase, pero sin implementar lógica: todos los métodos deben llevar pass.

No quiero que me expliques nada, solo pasa a Python lo que yo te voy a dar.

Clases que tengo:

1) Enums

- Color (WHITE, BLACK)

- MatchState (RUNNING, FINISHED)

2) Figure (abstracta)

Atributos:

- _x : int

- _y : int

- _color : Color

Métodos:

- +get_x(self) -> int

- +get_y(self) -> int

- +get_color(self) -> Color

- @abstractmethod

  - +can_move(self, x: int, y: int, board: Board) -> bool

- +is_enemy(self, other: Figure) -> bool

3) Piezas (todas heredan de Figure)

- Pawn:

  Atributos:

  - +has_moved : bool

  Métodos:

  - +can_move(self, x:int, y:int, board: Board) -> bool

- Rook:

  Atributos:

  - +has_moved : bool

  Métodos:

  - +can_move(self, x:int, y:int, board: Board) -> bool

- Knight:

  Métodos:

  - +can_move(self, x:int, y:int, board: Board) -> bool

- Bishop:

  Métodos:

  - +can_move(self, x:int, y:int, board: Board) -> bool

- Queen:

  Métodos:

  - +can_move(self, x:int, y:int, board: Board) -> bool

- King:

  Atributos:

- +has_moved : bool

Métodos:

- +can_move(self, x:int, y:int, board: Board) -> bool

4) Board (solo lectura)

Atributos:

- __figures : list[Figure]

Métodos:

- +get_figure_at(self, x: int, y: int) -> Figure | None
- +get_figure_count(self) -> int

5) MutableBoard (hereda de Board, modifica estado)

Métodos:

- +move(self, fig: Figure, x: int, y: int) -> None
- _set_figure_at(self, fig: Figure | None, x: int, y: int) -> None
- __add_figure(self, fig: Figure) -> None
- __remove_figure(self, fig: Figure) -> None
- +clear(self) -> None

6) Match (la partida)

Atributos:

- __id : int
- +board : MutableBoard
- +player1_name : str
- +player2_name : str
- +turn : Color
- +state : MatchState

Métodos:

- +get_id(self) -> int

- +get_turn(self) -> Color

- +get_match_state(self) -> MatchState

- +start(self) -> None

- +move(self, fig: Figure, x: int, y: int) -> bool

- _change_turn(self) -> None

- +is_in_check(self, color: Color) -> bool

- +can_castle(self, color: Color) -> bool

- +is_stalemate(self, color: Color) -> bool

7) MatchService (gestor de partidas)

Atributos:

- _matches : list[Match]

Métodos:

- +create_match(self, player1_name: str, player2_name: str) -> Match

- +get_match(self, match_id: int) -> Match | None

- +get_all_matches(self) -> list[Match]

- +move(self, match_id: int, fig: Figure, x: int, y: int) -> bool

- +delete_match(self, match_id: int) -> None

Ahora genera todo el código en Python cumpliendo todo lo que te he pasao, con pass en todos los métodos.

CODIGO

```python
from __future__ import annotations

from abc import ABC, abstractmethod
from enum import Enum


class Color(Enum):
    WHITE = "WHITE"
    BLACK = "BLACK"


class MatchState(Enum):
    RUNNING = "RUNNING"
    FINISHED = "FINISHED"


class Figure(ABC):
    def __init__(self, x: int, y: int, color: Color):
        self._x: int = x
        self._y: int = y
        self._color: Color = color

    def get_x(self) -> int:
        pass

    def get_y(self) -> int:
        pass

    def get_color(self) -> Color:
        pass

    @abstractmethod
    def can_move(self, x: int, y: int, board: Board) -> bool:
        pass

    def is_enemy(self, other: Figure) -> bool:
        return self.get_color() != other.get_color()


class Pawn(Figure):
    def __init__(self, x: int, y: int, color: Color):
        super().__init__(x, y, color)
        self.has_moved: bool = False
```

```
    def can_move(self, x: int, y: int, board: Board) -> bool:
        pass


class Rook(Figure):
    def __init__(self, x: int, y: int, color: Color):
        super().__init__(x, y, color)
        self.has_moved: bool = False

    def can_move(self, x: int, y: int, board: Board) -> bool:
        pass


class Knight(Figure):
    def __init__(self, x: int, y: int, color: Color):
        super().__init__(x, y, color)

    def can_move(self, x: int, y: int, board: Board) -> bool:
        pass


class Bishop(Figure):
    def __init__(self, x: int, y: int, color: Color):
        super().__init__(x, y, color)

    def can_move(self, x: int, y: int, board: Board) -> bool:
        pass


class Queen(Figure):
    def __init__(self, x: int, y: int, color: Color):
        super().__init__(x, y, color)

    def can_move(self, x: int, y: int, board: Board) -> bool:
        pass


class King(Figure):
    def __init__(self, x: int, y: int, color: Color):
        super().__init__(x, y, color)
        self.has_moved: bool = False

    def can_move(self, x: int, y: int, board: Board) -> bool:
        pass


class Board:
    def __init__(self):
        self.__figures: list[Figure] = []
```

```
    def get_figure_at(self, x: int, y: int) -> Figure | None:
        pass

    def get_figure_count(self) -> int:
        pass


class MutableBoard(Board):
    def __init__(self):
        super().__init__()

    def move(self, fig: Figure, x: int, y: int):
        pass

    def _set_figure_at(self, fig: Figure | None, x: int, y: int):
        pass

    def __add_figure(self, fig: Figure):
        pass

    def __remove_figure(self, fig: Figure):
        pass

    def clear(self):
        pass


class Match:
    def __init__(self, match_id: int, player1_name: str, player2_name: str):
        self.__id: int = match_id
        self.board: MutableBoard = MutableBoard()
        self.player1_name: str = player1_name
        self.player2_name: str = player2_name
        self.turn: Color = Color.WHITE
        self.state: MatchState = MatchState.RUNNING

    def get_id(self) -> int:
        pass

    def get_turn(self) -> Color:
        pass

    def get_match_state(self) -> MatchState:
        pass

    def start(self):
        pass

    def move(self, fig: Figure, x: int, y: int) -> bool:
        pass
```

```
    def _change_turn(self):
        pass

    def is_in_check(self, color: Color) -> bool:
        pass

    def can_castle(self, color: Color) -> bool:
        pass

    def is_stalemate(self, color: Color) -> bool:
        pass


class MatchService:
    def __init__(self):
        self._matches: list[Match] = []

    def create_match(self, player1_name: str, player2_name: str) -> Match:
        pass

    def get_match(self, match_id: int) -> Match | None:
        pass

    def get_all_matches(self) -> list[Match]:
        pass

    def move(self, match_id: int, fig: Figure, x: int, y: int) -> bool:
        pass

    def delete_match(self, match_id: int):
        pass
```